



Groupe 7N

HAI811I : Programmation mobile

Rapport

Liste des fonctionnalités implémentées et choix de conception



CANI Nicolas - DURAND Elliot

Faculté des Sciences
Université de Montpellier

18 Mai 2026

Table des matières

1	Introduction	3
2	Liste détaillée des fonctionnalités implémentées	3
2.1	Fonctionnalités communes à TravelShare et TravelPath	3
2.1.1	Authentification	3
2.1.2	Gestion du profil	3
2.2	Travel Share	4
2.2.1	Flux de publications	4
2.2.2	Groupes	6
2.2.3	Notifications	8
2.3	Travel Path	8
2.3.1	Création de parcours de visite	8
2.3.2	Consultation des parcours	9
2.3.3	Interaction avec les parcours	10
3	Architecture & Choix de conception	11
3.1	Architecture globale	11
3.2	Frontend	11
3.2.1	Choix de la stack	11
3.2.2	Choix de conception	11
3.2.2.1	Architecture réseau API RESTful	11
3.2.2.2	Architecture Single Activity	11
3.2.2.3	Patron d'architecture MVVM	11
3.2.2.4	Gestion de la mémoire et chargement d'images	12
3.2.2.5	Cartographie avec OpenStreetMap	12
3.2.2.6	Bonnes pratiques Android	12
3.2.2.7	Authentification	12
3.2.2.8	Centralisation de la logique pour la réutilisabilité	13
3.2.3	Passerelle entre TravelShare et TravelPath	13
3.3	Base de données	13
3.3.1	Choix de la stack	13
3.3.2	Schéma de la base de données	13
3.4	Backend	14
3.4.1	Choix de la stack	14
3.4.2	Sécurité	14
3.4.2.1	Les sécurités de base	14
3.4.2.2	Vérification des données textuelles en entrées	14
3.4.2.3	Vérification des fichiers en entrées	15
3.4.2.4	Authentification de l'utilisateur	15
3.4.2.5	Protection des informations de groupes	16
3.4.2.6	Autres Guards	17
3.4.3	Workflow complet d'une requête	17
3.4.4	Performance	17
3.4.4.1	Non envoi de ressource dans les DTO	17
3.4.4.2	Pagination	18
3.4.4.3	PostGIS	18
3.4.5	Scalabilité	18
3.4.5.1	Distribution des images	18
3.4.5.2	Utilisation des triggers SQL pour éviter les GROUP BY	19
3.4.6	Disponibilité	20
3.4.7	Portabilité	20
3.4.8	Maintenabilité et Évolutivité	20
3.4.9	Gestion des FcmToken	21
3.4.10	Algorithme de TravelPath	21
4	Features non implémentées et améliorations possibles	23
5	Conclusion	23
6	Lien du dépôt GitHub	23
	Bibliographie	24

1 Introduction

L'objectif de ce projet est de réaliser une application native Android réunissant deux applications en une : TravelShare et TravelPath. TravelShare serait une application de partage de photos de vacances, axée sur la dimension de groupes et de lieux (proche de l'application instagram). TravelPath serait lui un générateur de parcours intelligent de visite de ville, avec prises en compte de paramètres réels (météo, difficulté) et de préférences utilisateurs (type de lieux souhaités, budget et temps max.).

Nous expliquerons au long de ce rapport dans un premier temps le détail des fonctionnalités réalisées, sous forme de tableau. Nous expliquerons ensuite nos choix d'architectures et de conceptions. Nous prendrons ensuite du recul sur notre travail, avec les améliorations possibles, et finirons par une conclusion.

2 Liste détaillée des fonctionnalités implémentées

Légende :

- (*) : Disponible en mode « invité » (non connecté) et mode connecté.
- (**) : Réservé exclusivement à l'administrateur du groupe.
- *username* : Les données requises / envoyées sont optionnelles

2.1 Fonctionnalités communes à TravelShare et TravelPath

L'application est disponible en Anglais et en Français

2.1.1 Authentification

Fonctionnalité	Description	Données requises / envoyées
Inscription	Formulaire de création d'un compte	Email Nom d'utilisateur Mot de passe
Confirmation d'email	L'utilisateur doit confirmer son email en consultant l'email qu'il a reçu sur son email d'inscription	
Connexion	Formulaire de connexion	Email Mot de passe

2.1.2 Gestion du profil

Fonctionnalité	Description	Données requises / envoyées
Mise à jour de la photo de profil	Choix d'une image pour mettre à jour sa photo de profil	Photo
Mise à jour des informations personnelles	Mises à jour des informations de profils, si le mot de passe est modifié alors toutes les sessions actives sont supprimées	Email Nom d'utilisateur Mot de passe
Gestion des sessions actives	Consultation de tous les téléphones utilisant le compte, avec possibilité de suppression des sessions	Type d'appareil IP Date de création

Fonctionnalité	Description	Données requises / envoyées
Suppression du compte	Suppression du compte, avec toutes les données associées (postes, likes, etc..)	
Déconnexion	Déconnexion et suppression du FCM Token	

2.2 Travel Share

2.2.1 Flux de publications

Fonctionnalité	Description	Données requises / envoyées
Création de poste	Formulaire de création d'un poste	Photo Titre Localisation Type <i>Description</i> <i>Groupe associé</i> <i>Liste de tags</i> Pour certains types : <i>intervalle de durée, intervalle de prix</i>
Suggestion de tags	Le formulaire de création d'un poste est initialisé avec la liste des tags les plus populaires, fournit une autocomplétion pour les tags existants et une suggestion de tag par IA (appel à Gemini avec une clé API)	Liste de tags
Modification de poste	Formulaire de modification d'un poste	<i>Titre</i> <i>Type</i> <i>Description</i> <i>Pour certains types : intervalle de durée, intervalle de prix</i>
Suppression de poste	Bouton de suppression avec confirmation pour supprimer un post (et tout ce qui lui est associé)	
(*) Consultation des postes sur un flux infini	Flux de postes sous forme de liste. Pagination avec refresh tous les 20 postes	Photo Titre <i>Description</i> Date Localisation Nombre de likes / commentaires Type <i>Pour certains types : intervalle de durée, intervalle de prix</i> <i>Tags</i> Utilisateur associé (nom d'utilisateur et photo de profil) <i>Groupe associé</i> <i>Audio</i>
(*) Recherche poussée de postes	Formulaire de recherche par : - Titre, Description, utilisateur ou groupe - Type de publication	Pareil que ci dessus

Fonctionnalité	Description	Données requises / envoyées
	- Liste de tag (et logique entre les tags) - Localisation avec lieu exact et rayon de distance (5, 10, 20, 50, 100, 200 et 500 km)	
(*) Carte interactive avec les postes	Carte OpenStreetMap avec affichage des postes sous forme de point de navigation (noir). Nouvelle requête de manière intelligente (dès le changement de position ou de rayon) pour éviter de surcharger la carte. Possibilité de cliquer sur les points pour voir les postes	Carte avec points de navigation
(*) Consultation des commentaires	Liste des commentaires et de leurs réponses	Contenu du commentaire Utilisateur associé <i>Si le commentaire a été modifié</i> Date Nombre de likes <i>Nombre de réponses</i> <i>Indentation si le commentaire est une réponse</i>
(*) Ajout et retrait de like sur le posts et les commentaires	Cliquer met un like et recliquer l'enlève	
(*) Consultation de la localisation précise avec redirection	Affichage des coordonnées du poste, de la date précise et de bouton de redirection vers Google Maps et TravelPath	Coordonnées Date Intent
Sauvegarde des postes via les signets	Cliquer met un signet et recliquer l'enlève	
Ajouter un commentaire	Input de texte pour ajouter un commentaire associé à un post, ou réponse à un commentaire existant	Contenu du commentaire
Modification de commentaire	Bouton permettant de modifier le contenu d'un commentaire	Contenu du commentaire
Suppression de commentaire	Bouton de suppression avec confirmation pour supprimer un post	
Signalement d'un poste	Formulaire de signalement associé à un poste	Raison <i>Détails</i>

Fonctionnalité	Description	Données requises / envoyées
(*) Consultation de la fiche utilisateur associée à un post	Clic sur la photo de profil ou le nom d'utilisateur → fiche affichée avec : nombre de posts, nombre de groupes rejoints, abonnés et abonnements. Également son flux de derniers postes. En mode connecté : cloche d'abonnement à l'utilisateur	Infos utilisateur (nombre de posts, nombre de groupes, abonnés et abonnements) Flux de posts associés
(*) Consultation de la fiche tag associée à un post	Clic sur un tag → fiche affichée avec : nombre d'utilisations, émoji flamme si top 5 des plus utilisés. En mode connecté : cloche d'abonnement au tag	Infos tag (nombre d'utilisations) Flux de posts associés
(*) Consultation de la fiche lieu associée à un post	Clic sur le lieu → fiche affichée avec : nombre d'utilisations, émoji flamme si top 5 des plus utilisés. En mode connecté : cloche d'abonnement au lieu	Infos lieu (nombre d'utilisations) Flux de posts associés

2.2.2 Groupes

Fonctionnalité	Description	Données requises / envoyées
Création de groupe	Formulaire de création de groupe	<i>Photo de profil du groupe</i> Nom Si le groupe est privé <i>Description</i>
Liste de mes groupes	Affichage des groupes membres sous forme de liste	Nom <i>Photo de profil du groupe</i> <i>Description</i> Date Si le groupe est privé Nombre de membres Nombre de postes associés
Recherche de groupe par nom	Affichage des groupes recherchés sous forme de liste	Pareil que ci dessus
Groupes populaires	Affichage des groupes les plus populaires par nombre de membres sous forme de liste	Pareil que ci dessus
Fiche du groupe	- Groupe public : Bouton rejoindre immédiatement et postes associés consultables - Groupe public et membre : Informations du groupe, liste des membres, postes associés et cloche de notification	Informations du groupe <i>Liste des membres (nom d'utilisateur et photo de profil)</i> Flux de postes

Fonctionnalité	Description	Données requises / envoyées
	- Groupe privé : Bouton demande d'adhésion - Groupe privé et membre : Informations du groupe, liste des membres, postes associés et cloche de notification	
Quitter un groupe	Le membre peut quitter le groupe en cliquant sur le bouton associé dans la fiche de groupe	
(**) Bannissement / Transfert des droits d'administration	En consultant la liste des membres, l'administrateur du groupe peut bannir ou transférer le rôle d'administrateur à un des membres	
(**) Validation ou refus des demandes d'adhésion	En consultant la liste des demandes d'adhésion, l'administrateur du groupe peut valider ou refuser la demande d'adhésion d'un utilisateur	
(**) Débannissement	En consultant la liste des bannissements, l'administrateur du groupe peut révoquer le bannissement d'un utilisateur	
(**) Modération : suppression de n'importe quelle publication du groupe	L'administrateur peut supprimer n'importe quel poste en consultant le flux de postes associé à son groupe	
(**) Modification du groupe	Formulaire de modification du groupe	<i>Nom Si le groupe est privé Description</i>
(**) Mise à jour de l'avatar du groupe	Choix d'une image pour mettre à jour la photo de profil du groupe	<i>Photo de profil du groupe</i>
(**) Suppression du groupe	Suppression du groupe (nécessite la suppression préalable de tous les posts)	

2.2.3 Notifications

Fonctionnalité	Description	Données requises / envoyées
Abonnement aux publications d'un utilisateur, d'un groupe, d'un lieu ou d'un tag spécifique	Comme décrit ci dessus, les utilisateurs peuvent s'abonner et recevoir une notification lors d'un nouveau post associé à leurs abonnements	Notification Android
Notification lors d'un like sur un poste personnel	L'utilisateur recevra une notification lors d'un like sur un de ses posts	Notification Android
Consultation de la liste des notifications	Flux de notifications sous forme de liste avec possibilité de cliquer sur le poste associé et consulter sa fiche	Type de notification Poste associé <i>Utilisateur/Groupe/Lieu/Tag concerné</i>
Liste de tous les abonnements	Liste de toutes les entités auxquelles l'utilisateur est abonné avec possibilité de se désabonner en cliquant sur le bouton associé	Type d'abonnement <i>Utilisateur/Groupe/Lieu/Tag concerné</i>
Badge indiquant le nombre de notifications	Badge rouge sur l'icône de notifications, située dans la barre de navigation, indiquant le nombre de nouvelles notifications	Numéro

2.3 Travel Path

2.3.1 Création de parcours de visite

Fonctionnalité	Description	Données requises / envoyées
Création de parcours	Formulaire de création de parcours	Ville / lieu de départ <i>Type(s) de lieu préférés</i> Budget maximal Durée maximale Moment de la journée (matin, midi, après midi, soir) Mode de transport (à pied, voiture)
Sélection du parcours	Liste des parcours proposés (Normal ou Buisness) et bouton de confirmation	Ville / lieu de départ Météo Type du parcours (normal, buisness) Temps estimé Budget estimé Nombre d'étapes Ville de départ Carte interactive avec chaque étape Pour chaque étape : numéro de l'étape, temps de trajet estimé pour s'y rendre (symbole de

Fonctionnalité	Description	Données requises / envoyées
		certification par API), temps sur place, prix sur place, type, lieu, poste associé (tous les éléments sont cliquables)
Regénération de parcours	Bouton pour régénérer les parcours	Nouveaux parcours (pareil que ci dessus)

2.3.2 Consultation des parcours

Fonctionnalité	Description	Données requises / envoyées
(*) Consultation des parcours sur un flux infini	Flux de parcours sous forme de liste	Météo Type du parcours Moment de la journée (Matin, Midi, Après midi, Soir) Mode de transport (À pied, Voiture) Temps estimé Budget estimé Nombre d'étapes Ville de départ Carte interactive avec chaque étape Pour chaque étape : numéro de l'étape, temps de trajet estimé pour s'y rendre (symbole de certification par API) avec distance et heure estimée d'arrivée, temps sur place, prix sur place, type, lieu, poste associé (tous les éléments sont cliquables)
(*) Recherche de parcours par géographie	Formulaire de recherche par : Localisation avec lieu exact et rayon de distance (5, 10, 20, 50, 100, 200 et 500 km)	Pareil que ci dessus
(*) Carte interactive avec les parcours	Carte OpenStreetMap avec affichage des parcours sous forme de point de navigation (orange). Nouvelle requête de manière intelligente (dès le changement de position ou de rayon) pour éviter de surcharger la carte. Possibilité de cliquer sur les points pour voir les parcours	Carte avec points de navigation
Consulter mes parcours	Flux de parcours sous forme de liste, uniquement les miens	Pareil que ci dessus

Fonctionnalité	Description	Données requises / envoyées
Sauvegarde des parcours via exportation en PDF	Bouton pour exporter en PDF le parcours en bas de l'écran de consultation des différentes étapes du parcours	Pareil que ci dessus au format PDF

2.3.3 Interaction avec les parcours

Fonctionnalité	Description	Données requises / envoyées
Ajout et retrait de like sur le parcours	Cliquer met un like et recliquer l'enlève	
Sauvegarde des parcours via les signets	Cliquer met un signet et recliquer l'enlève	

3 Architecture & Choix de conception

3.1 Architecture globale

Ci dessous un schéma d'architecture globale :

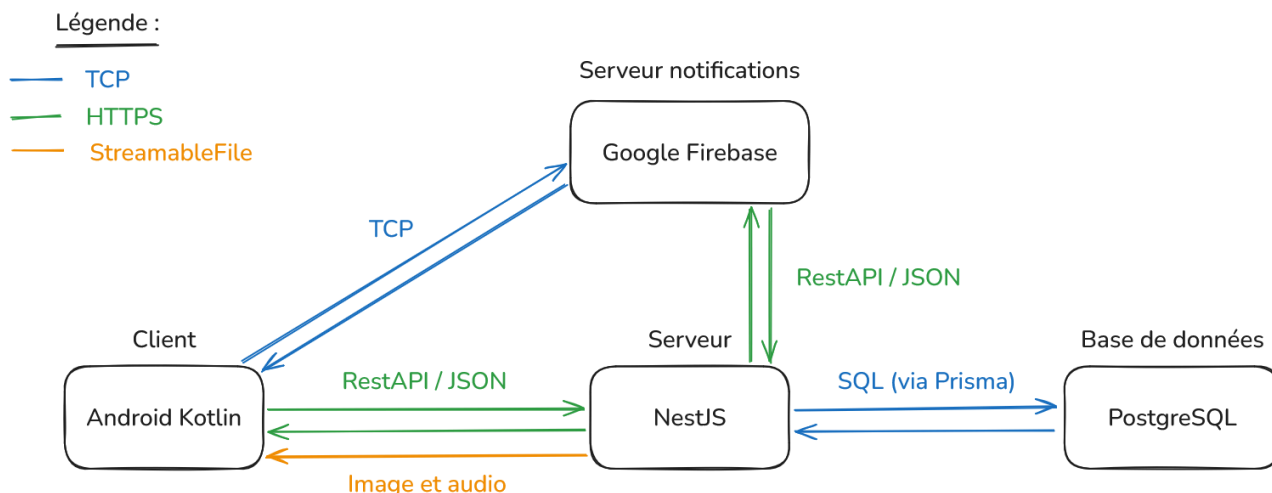


Fig. 1. – Schéma d'architecture globale

3.2 Frontend

3.2.1 Choix de la stack

Nous avons fait le choix de réaliser l'application en Kotlin [1], ce langage étant plus puissant pour les applications modernes Android, surtout depuis son adoption officielle par Google.

3.2.2 Choix de conception

3.2.2.1 Architecture réseau API RESTful

Notre application communiquera exclusivement avec le service web Rest, qui est aujourd'hui le choix standard. Nous avons choisi cette technologie pour la simplicité de mise en oeuvre. Les alternatives comme gRPC ou GraphQL présentent d'excellentes performances, cependant les gains restent négligeables par rapport à la complexité d'implémentation et du temps que nous avons.

3.2.2.2 Architecture Single Activity

L'architecture Single Activity est implémentée. Une seule activité hôte (MainActivity) nous sert de conteneur alors que les différents écrans sont matérialisés par des fragments. Cette approche moderne évite le modèle historiquement lourd de Android, avec une activité correspondant à un seul écran, qui est bien plus gourmand en ressource.

Grâce à cela, nous pouvons simplifier le partage de données (par exemple notre jeton d'authentification) et fluidifier la navigation entre nos deux applications interconnectées, TravelShare et TravelPath.

3.2.2.3 Patron d'architecture MVVM

Le projet utilise le patron MVVM pour séparer proprement les responsabilités :

- Model : C'est la couche de données. Elle comprend les data classes Kotlin (les DTO) qui structurent les réponses de l'API, et la couche réseau (ApiClient)
- View : Ce sont les Fragments. Ils gèrent uniquement l'affichage et les clics de l'utilisateur, sans embarquer de logique complexe

- **ViewModel** : Ce sont les classes comme `AuthViewModel`. Ils font le lien entre le Model et la View. Ils stockent l'état de l'interface (ce qui évite de perdre les données si l'écran tourne) et notifient les fragments via des flux observables (`LiveData`)

3.2.2.4 Gestion de la mémoire et chargement d'images

Notre backend nous envoie les images sous forme de flux binaire (`StreamableFile`), la gestion des images côté client impose ainsi une certaine rigueur afin de garantir performance, consommation moindre et empêcher le crash de l'application avec un memory bound. La bibliothèque `Coil` a ainsi été utilisée afin de résoudre cette problématique (surement la plus importante du projet), ce choix a été motivé par la capacité de `Coil` à nativement :

- Déporter le téléchargement des images de manière asynchrone dans un pool de threads (performance maximum)
- Analyser les dimensions de `ImageView` et redimensionnement en mémoire à la taille exacte requise (aucun gaspillage de RAM)
- Utiliser un double cache, avec un cache mémoire et un cache persistant (performance et économie d'énergie avec limitation des requêtes HTTP)

3.2.2.5 Cartographie avec OpenStreetMap

Pour l'affichage des cartes interactives (visualisation des postes de `TravelShare` et des étapes de parcours dans `TravelPath`), le choix s'est porté sur `OpenStreetMap` via la bibliothèque open-source `osmdroid` et son composant graphique `org.osmdroid.views.MapView`. Cette solution a été préférée à l'alternative standard `Google Maps` pour plusieurs raisons techniques et pragmatiques :

- Contrairement au SDK de Google, `OpenStreetMap` offre une gratuité totale sans nécessiter l'activation d'un compte de facturation cloud. De plus, son indépendance vis-à-vis des `Google Play Services` élimine toute télémétrie ou processus d'arrière-plan caché, ce qui préserve l'autonomie.
- Le rendu cartographique étant une opération lourde, le composant `MapView` est explicitement synchronisé avec le cycle de vie des fragments (`onResume()` et `onPause()`). Lors de la destruction de la vue (`onDestroyView()`), l'appel à la méthode `onDetach()` d'`osmdroid` force la coupure totale du rendu. La référence est ensuite passée à null pour permettre au Garbage Collector de libérer immédiatement la RAM.

Cette implémentation légère répond parfaitement aux exigences de notre logique de requêtage dynamique par rayon de distance pour l'affichage des points de navigation.

3.2.2.6 Bonnes pratiques Android

- N'utiliser que les ressources, pour les textes, les couleurs ou les styles
- Utilisation des `ConstraintLayout` partout, ainsi que des `RecyclerView` pour l'affichage des listes, tout en utilisant `Material Components`
- Utilisation du fichier `local.properties` comme stockage des variables d'environnement
- Utilisation d'Adapters l'affichage des listes
- Abonnement et désabonnement (cycle de vie avec `OnResume()` et `OnPause()`) propre lors d'utilisation de capteur (localisation et appareil photo)

3.2.2.7 Authentification

- Centralisation et stockage sécurisé des tokens d'authentification dans la classe `SessionManager`. Cette classe sauvegarde de manière chiffrée et permanente le token grâce à la bibliothèque `AndroidX Security`, via `EncryptedSharedPreferences` et sa classe `MasterKey`. `EncryptedSharedPreferences` permet de gérer simplement un couple clé-valeur chiffré, sans la complexité d'une base de données `SQLite`.

- La classe `AuthViewModel`, qui étend la classe abstraite `ViewModel`, expose une variable `_isLoggedIn` de type `MutableLiveData<Boolean>`, ce qui permet à n'importe quel fragment d'observer la propriété `isLoggedIn` et de savoir si l'utilisateur est connecté. Cela permet d'afficher conditionnellement les informations : par exemple, si l'utilisateur n'est pas connecté, on affiche la page de connexion à la place de la page de notifications.

3.2.2.8 Centralisation de la logique pour la réutilisabilité

- Centralisation des méthodes génériques réutilisables dans le package `utils`. Par exemple, les appels API Rest sont centralisés dans la classe statique `ApiClient`, avec une méthode statique par méthode HTTP (GET, POST, PATCH, DELETE). Chaque méthode log l'entrée et la sortie, cette manière de faire nous a permis de facilement résoudre les bugs
- Utilisation d'intercepteur pour rajouter silencieusement le token d'authentification (ou token anonyme) pour chaque appel d'une méthode de la classe `ApiClient` (en utilisant la classe `SessionManager`)
- Utilisation des `Helpers` afin de centraliser la logique des actions associés aux ressources tels que les posts, les parcours, etc. Par exemple, on ne code qu'une fois le fait qu'un like sur un post déclenche l'appel API `POST /like/post` avec l'id du post, et cette logique est réutilisée à chaque endroit où on affiche des postes (feed, bookmarks, etc)

3.2.3 Passerelle entre TravelShare et TravelPath

L'interconnexion des deux services se fait naturellement dans la même application. Sur la page d'accueil, il suffit de cliquer en haut à droite pour passer de l'affichage des postes à celui des parcours. La recherche intègre également une séparation distincte entre les deux entités mais elles sont sur la même page, les signets fonctionnent également de la même manière. Nous avons une séparation nécessaire pour les distinguer mais il est très facile de passer de l'un à l'autre. Notre carte interactive affiche également les deux types d'entités sous forme d'icônes de couleurs différentes (noir pour TravelShare et orange pour TravelPath).

Cependant la passerelle la plus forte est le fait que TravelPath utilise exclusivement les postes de TravelShare pour les étapes de ses parcours. Cette implémentation nous a permis d'enrichir les postes de TravelShare (nous avons intégré l'intervalle de temps et de prix), de donner un vrai sens au type du poste, mais également de travailler sur nos propres données pour l'algorithme de TravelPath (et ne pas faire d'appel API externe pour construire le parcours).

3.3 Base de données

3.3.1 Choix de la stack

La base de données choisie est PostgreSQL [2], c'est actuellement la référence des bases de données relationnelles et notre projet ne nécessite pas de NoSQL. De plus, nous avons utilisé PostGIS [3], une extension de Postgre, permettant des calculs de distances directement depuis le SGBD, maximisant ainsi la performance et allégeant la logique métier.

Le choix de l'ORM est Prisma [4], qui est un des choix de référence pour du typescript, il est également open source. Il présente l'avantage de définir la structure des tables dans des fichiers de schéma dédiés, avec une syntaxe légère mais expressive. Il permet d'écrire des requêtes concises, et gère nativement les relations entre tables, évitant d'avoir à écrire des requêtes SQL longues et illisibles. La commande `prisma studio` permet d'accéder à une interface graphique ressemblant à PhpMyAdmin, cette fonctionnalité est absolument essentielle. Son écosystème est très poussé, par exemple la bibliothèque `prisma-erd-generator` [5], nous permet de générer un schéma relationnel avec la commande `npx prisma generate`.

3.3.2 Schéma de la base de données

Ci dessous le schéma de la base de données (il faut zoomer et l'image sera nette) :

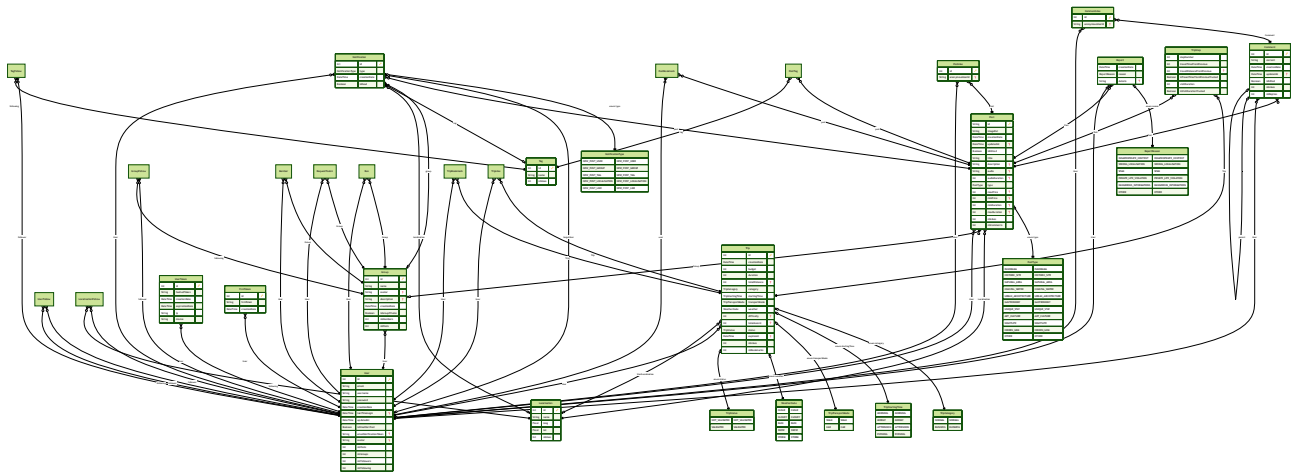


Fig. 2. – Schéma de la base de données

Les énumérations sont dédoublées, ceci est un bug de prisma-erd-generator, dans la réalité il n'y a pas de duplication de valeurs.

3.4 Backend

3.4.1 Choix de la stack

Nous avons choisi NestJS pour le backend. Ce framework open source intègre nativement Typescript, comblant le plus gros défaut du Javascript, tout en offrant sa flexibilité et sa gigantesque bibliothèque gérée par NPM. NestJS impose une architecture robuste (Architecture en couche avec pattern Controller-Service-Repository et approche orientée objet) et très standardisée, permettant de très rapidement mettre en place une API Rest performante et sécurisée. Il possède également une très bonne documentation et pleins d'outils nous permettant de mettre en place rapidement tout ce dont nous avons besoin.

Les aspects non fonctionnels ont été très fortement considérés, particulièrement sur la sécurité, la performance et la scalabilité.

3.4.2 Sécurité

3.4.2.1 Les sécurités de base

La documentation de NestJS montre comment facilement mettre en place des sécurités de base.

Cette page [6] nous permet d'activer Helmet, qui est un middleware qui ajuste les headers de réponse HTTP pour éviter certains types d'attaques courantes, en seulement une ligne de code.

Cette page [7] nous permet d'activer un rate limit, permettant ainsi de se protéger contre les requêtes abusives ou les attaques brute force. Nous avons laissé les paramètres par défauts, qui sont suffisants.

Toutes les informations sensibles sont hachées. Les mots de passe utilisateurs utilisent la bibliothèque bcrypt [8]. Les tokens d'authentification sont quant à eux hachés en SHA256. Les tokens étant des valeurs aléatoires de 128 bits, il est impossible de les deviner par force brute ou de construire une rainbow table efficace contre eux. SHA256 est donc suffisant et bien plus rapide que bcrypt, ce qui est indispensable car chaque requête utilisateur nécessite le rehachage de son token pour vérification.

3.4.2.2 Vérification des données textuelles en entrées

La plupart des requêtes nécessitant la création ou la modification d'une ressource prennent en entrées des données textuelles (DTO) ou un / des fichiers (image ou audio).

La vérification de chaque attribut d'un DTO est vérifiée automatiquement grâce à la bibliothèque `class-validator`. Comme indiqué dans la documentation [9], il suffit de décorer un de nos attributs dans les DTO pour faire automatiquement la vérification, nous pouvons même personnaliser le message renvoyé en cas de non respect du format demandé. Par exemple le décorateur `@IsEmail()` vérifie automatiquement le bon format d'un email, sans avoir besoin d'écrire du regex. Ci dessous un exemple pour la vérification de notre mot de passe :

```
@IsNotEmpty()  
@IsStrongPassword(  
  { minLength: 6, minSymbols: 0, minUppercase: 0 },  
  { message: "Password must be at least 6 characters long with 1 number" }  
)  
password!: string;
```

Liste 1. – Extrait d'un de nos DTO

Nous pouvons aisément comprendre que le mot de passe exige qu'il ne soit pas vide (`@IsNotEmpty()`) avec au moins 6 caractères et un seul nombre (`minSymbols: 0` n'exigeant pas de caractères spéciaux et `minUppercase: 0` aucune majuscule). Le message d'erreur renvoyé est personnalisé.

Cette approche de la validation des DTO à l'entrée nous permet de séparer proprement les responsabilités et automatise le processus de vérification. Il est une pratique essentielle dans le développement aujourd'hui.

3.4.2.3 Vérification des fichiers en entrées

La vérification des fichiers se fait à l'aide de la bibliothèque Multer, la documentation [10] explique encore une fois très facilement comment mettre en place un système robuste de vérification, avec des pipes personnalisés. Dans notre cas, nous limitons les types de fichiers acceptés (par exemple png et jpeg uniquement pour les images) et la taille (max. 5 Mo).

3.4.2.4 Authentification de l'utilisateur

Nous avons fait le choix de partir sur une authentification côté serveur. Les JWT utilisés côté client, présentent de nombreuses failles de sécurité s'ils sont implémentés sans sécurité supplémentaire (par exemple pas de mécanisme de access / refresh token). L'authentification serveur est plus sécurisée et elle permet à l'utilisateur de gérer ses sessions, ce que notre application propose.

Le choix de base se portait sur les sessions express [11], cependant ce type d'authentification n'est pas idéal pour les applications Android, car elles ne gèrent pas nativement les cookies comme un navigateur web. Nous avons ainsi opté pour un système d'authentification basé sur l'architecture des sessions express mais adapté à notre cas d'usage.

Lors de la connexion, le serveur génère un token aléatoire (bibliothèque `crypto`) en clair et haché, il l'envoie à l'utilisateur (qui le stocke dans son `SessionManager`) celui en clair et celui haché sont stockés en base de données, associé au `userId` (via l'intercepteur). L'utilisateur enverra son token dans chaque requête HTTP. Le but du serveur est de rehasher ce token envoyé et trouver une correspondance en base de données.

NestJS permet d'utiliser des « Guards » [12], l'équivalent des middleware, ces derniers s'exécutent avant la l'exécution du service (la logique métier). Elle centralise la logique d'authentification sans polluer le code métier (principe SoC). Dans notre projet, chaque route nécessitant une connexion de l'utilisateur utilise la Guard « `AuthGuard` ». Ci dessous un schéma illustrant cette guard :

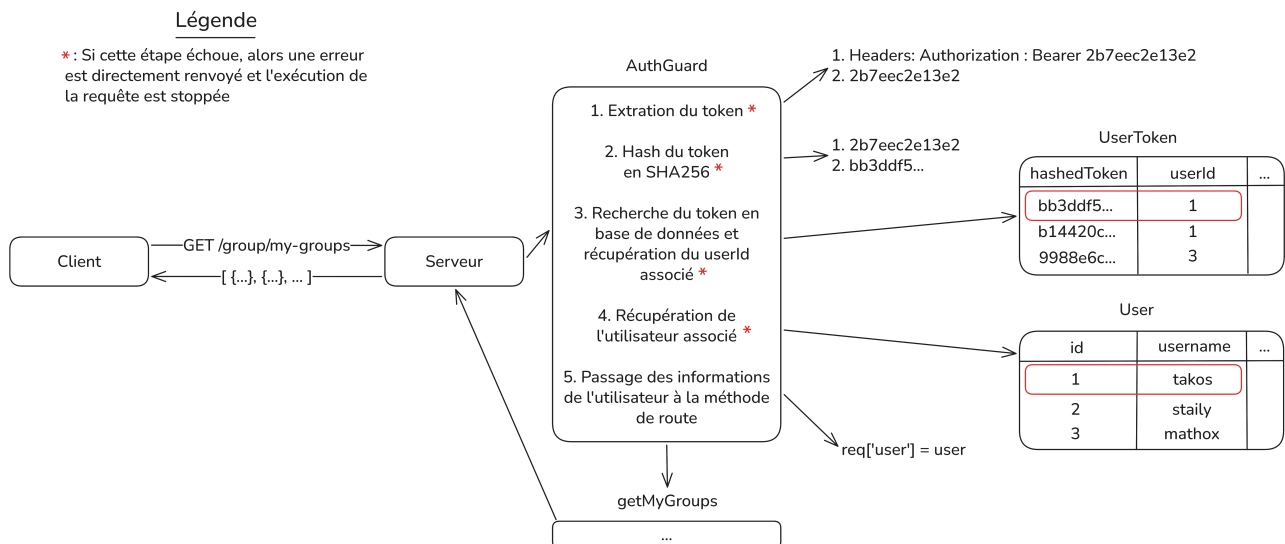


Fig. 3. – Schéma illustrant AuthGuard

Ce schéma illustre le flow d'un l'identification de l'utilisateur faisant la requête. Comme mentionné précédemment, tous les tokens sont hachés, ce qui empêche leur utilisation malveillante en cas vol de la base de données.

La logique de chiffrement du token et de récupération de l'utilisateur associé au token sont des méthodes externes conçues pour être réutilisées par d'autres guard (nous verrons cela juste après).

La guard transfère également toutes les informations utiles de l'utilisateur tels que l'id, le username ou l'email au service grâce à req['user']. Un décorateur NestJS [13] permet de récupérer ces informations facilement, pour un code robuste et réutilisable.

3.4.2.5 Protection des informations de groupes

Dans notre application, certains groupes sont privés, il est alors primordial de sécuriser toutes les routes API donnant accès à ces informations. Le but est de centraliser la logique de vérification d'accès à n'importe quelle ressource d'un groupe privé (photo, commentaire d'un poste, audio, etc...), dans une seule et même Guard. Ci dessous un schéma illustrant cette guard :

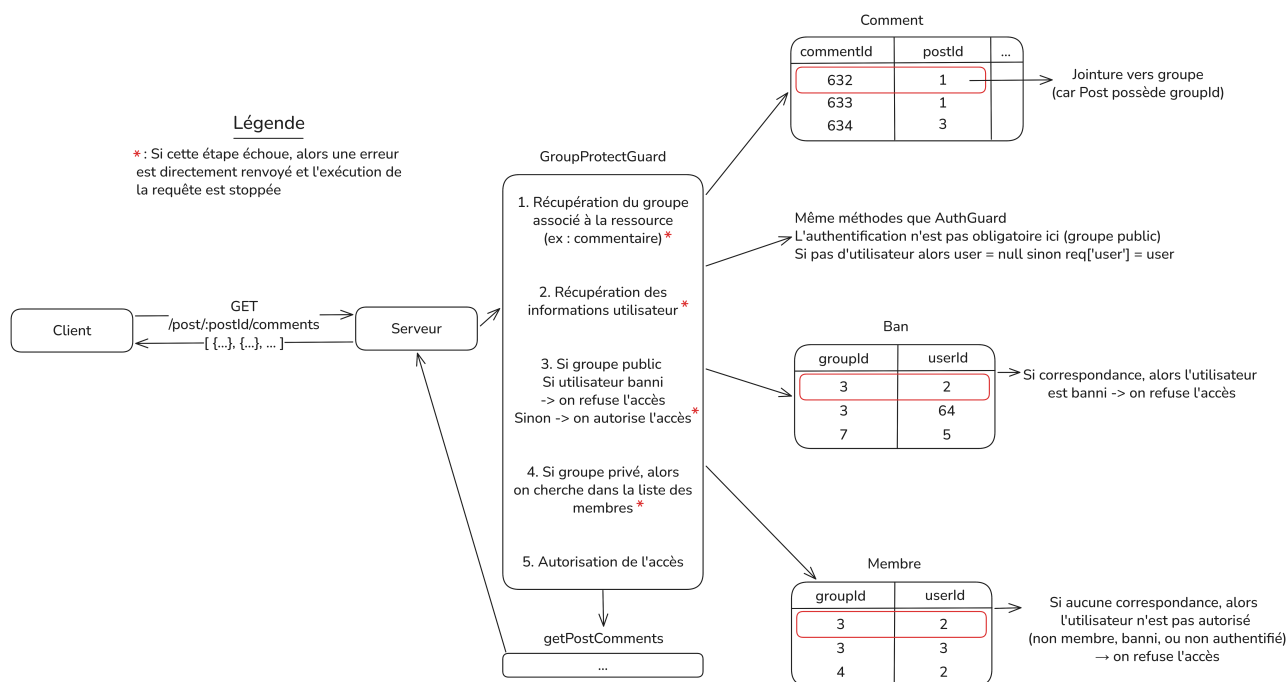


Fig. 4. – Schéma illustrant GroupProtectGuard

3.4.2.6 Autres Guards

Notre application utilise la Guard GroupAdminGuard, qui permet de certifier qu'une requête exigeant que l'utilisateur soit l'administrateur du groupe est bien ce dernier. Elle est toujours activée après la AuthGuard, récupère les informations de l'utilisateur (les Guard peuvent se passer des informations entre elles), et cherche s'il est bien administrateur du GroupId ciblé. Pas de grande complexité ici il s'agit d'une simple requête à la base de données.

La dernière Guard utilisée est AuthOptionalGuard, qui force l'utilisation d'un token anonyme si l'utilisateur n'est pas connecté, afin d'identifier un like et de pouvoir le retirer.

3.4.3 Workflow complet d'une requête

Ci dessous un schéma décrivant le workflow complet d'une requête dans notre architecture [14] :

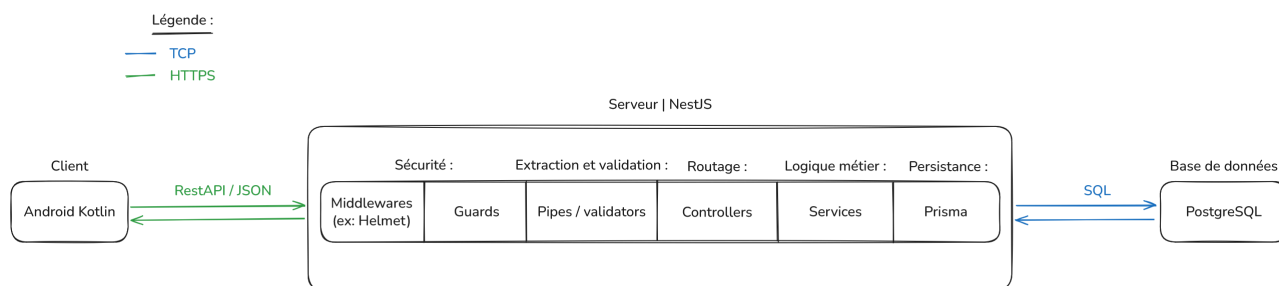


Fig. 5. – Schéma illustrant le workflow complet d'une requête

Nous avons là un aperçu visuel de la puissance de l'architecture NestJS avec le respect des principes SoC, de modularité et de maintenabilité du code.

3.4.4 Performance

3.4.4.1 Non envoi de ressource dans les DTO

Le non envoi de ressource dans les DTO de réponses si non présent. Par exemple si un poste n'appartient pas à un groupe, alors les données telles que groupId, groupAvatar et groupName ne sont pas envoyées

avec une valeur null, elles ne sont pas envoyées du tout. Ceci allège les DTO et multiplié par le nombre d'informations envoyées, cela augmente les performances.

3.4.4.2 Pagination

Pour un scroll infini ou bien une longue liste d'éléments, il est important de par exemple ne pas envoyer la liste complète, mais de la découper en petits segments et envoyer seulement une nouvelle portion (dans notre cas de 20) lorsque l'utilisateur le demande. Le terme technique est la pagination, cela permet au téléphone de ne pas charger trop de données en mémoire. En effet, si notre application a 200 posts publics, sans pagination, l'utilisateur devrait attendre de recevoir cette liste, alors que avec pagination, il devra attendre 20 posts, soit x10 plus rapide. Cela économise également de la bande passante et réduit la temps de calcul du serveur.

3.4.4.3 PostGIS

Comme mentionné dans notre choix de la base de données, PostGIS est une extension idéale pour le calcul géographique. Grâce à un index spatial (invisible dans les schemas prisma) de type GIST, chaque localisation sauvegardée est stockée sous forme d'arbre spatial (R-TREE [15]). Dès lors qu'une requête demande un calcul géographique (obtenir les localisations proches), nous utilisons la condition `ST_DWithin` dans la requête avec la longitude et la latitude de notre point de départ. Cette implémentation maximise les performances (calcul délégué au SGBD), respecte le principe SoC et utilise les dernières normes pour le calcul géographique, qui est au cœur de notre projet.

3.4.5 Scalabilité

3.4.5.1 Distribution les images

Pour le moment, la distribution des images s'effectue grâce à NestJS [16], cette méthode fonctionne très bien et permet de protéger les informations et médias en lectures pour les groupes privés. Cependant, plus l'application aura d'utilisateurs, plus le streaming de fichier empêchera de bonnes performances, car cela monopoliserait trop de bande passante.

L'application est conçue pour très rapidement et facilement implémenter la délégation de la distribution d'image à un bucket S3. Il suffira de modifier les méthodes de la classe `CdnService`, ce service implémente le pattern Adapter pour la résolution dynamique d'URL d'image ou d'audio. Dans notre cas nous n'utilisons pas de bucket, mais cela démontre la capacité à concevoir une application facilement scalable. Ci dessous un schéma montrant l'avant après, avec l'exemple de la méthode `getPostUrl` de la classe `CdnService` :

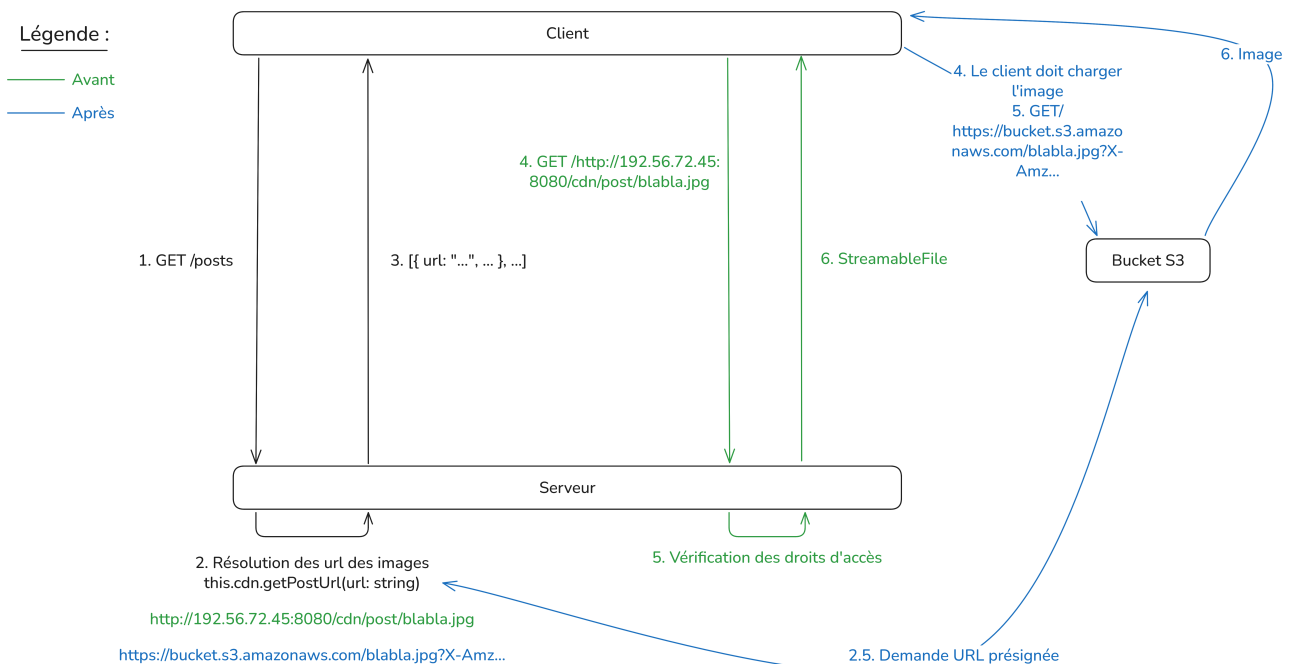


Fig. 6. – Schéma illustrant L'avant après de la résolution dynamique d'URL des images

Même si ce n'est pas explicité sur ce schéma, une vérification de droits d'accès se fait déjà au niveau de la requête GET /posts (GroupProtectGuard dans Fig. 4). Donc même en utilisant un S3, jamais un utilisateur n'ayant pas les droits de visualiser cette image ne recevra d'URL présignée, garantissant ainsi la sécurité des données.

3.4.5.2 Utilisation des triggers SQL pour éviter les GROUP BY

Beaucoup de relations sont en Many to One et Many To Many sont présents dans notre projet, par exemple la table like ou bien commentaire.

Lorsque l'on affiche les postes, il faut envoyer le nombre de likes et de commentaires du post, la méthode classique serait de faire des COUNT(*) avec des jointures sur les tables likes et commentaires avec des GROUP BY. Cette approche fonctionne, mais elle complexifie l'écriture des requêtes SQL dans le code métier, et force le SGBD à calculer des jointures, des GROUP BY et des COUNT. Sur un nombre conséquent d'utilisateurs, cela peut avoir un impact.

L'approche scalable est d'utiliser les triggers SQL. Prenons l'exemple des likes, la table Post possède désormais un attribut nbLikes, qui est incrémenté et décrémenté par un trigger lors d'une opération d'insertion ou de déletion sur la table Like. Ci dessous un extrait du code du trigger :

```

[ ... ]

IF (TG_OP = 'INSERT') THEN
    UPDATE "Post" SET "nbLikes" = "nbLikes" + 1 WHERE id = NEW."postId";
ELSIF (TG_OP = 'DELETE') THEN
    UPDATE "Post" SET "nbLikes" = "nbLikes" - 1 WHERE id = OLD."postId";

[ ... ]
CREATE TRIGGER tr_post_like_count AFTER INSERT OR DELETE ON "Like" FOR EACH ROW EXECUTE
FUNCTION update_post_like_count();

```

Liste 2. – Extrait d'un de nos triggers

Avec ceci, nous n'avons aucune jointure inutile à faire lors d'une requête de consultation et pouvons simplement sélectionner l'attribut. De plus, la séparation des responsabilités est accentuée, le code métier n'a pas à gérer l'incrémentation ou décrémentation lors d'une insertion ou déletion de like, le trigger agit silencieusement au plus bas niveau, donc de la manière la plus performante possible. Nous n'avons également pas à gérer les erreurs lors d'une transaction, car le trigger s'exécute dans la même transaction que le INSERT ou DELETE, et est automatiquement rollbacké en cas d'échec.

Nous avons pris l'exemple des likes, mais cette logique est implémentée dans notre application pour d'autres usages, par exemple compter le nombre de postes d'un utilisateur, le nombre de commentaires d'un poste, le nombre de membres d'un groupe ou encore le nombre d'utilisations des différents tags ou lieux de publications.

Pour être objectif, cette approche n'est pas parfaite : elle a un coût supplémentaire sur les opérations d'écriture. Elle apporte néanmoins plus de bénéfices en termes de simplicité du code et de performances en lecture, d'autant plus que la grande majorité des utilisateurs de ce type d'applications sont des spectateurs et n'interagissent pas activement avec les postes.

3.4.6 Disponibilité

NestJS intègre nativement un filtre global capturant toutes les exceptions non gérées et renvoyant une erreur 500 standardisée [17]. Cette fonctionnalité garantit qu'en cas d'erreur interne imprévue, le serveur reste en ligne et continue de traiter les requêtes des autres utilisateurs.

Nous le verrons après mais nous utilisons deux services d'API tierces. Les appels à ces API se font de manière à respecter le principe ouvert / fermé et SoC, ces derniers sont isolés dans des méthodes privées, encapsulées par des try catch. En cas d'erreur, le format de renvoi étant fortement typé, nous renvoyons une valeur standardisée tout en précisant au frontend que ces données sont des données non certifiées. De cette manière, notre application a une forte résilience face aux appels API externes.

3.4.7 Portabilité

Afin de pouvoir déployer le projet dans n'importe quel environnement, la conteneurisation de ce dernier est indispensable. Après avoir créé notre Dockerfile [18], nous avons créé un Docker Compose [19] afin de construire et lancer automatiquement notre application ainsi que sa base de données. Cette dernière est dotée d'un volume, assurant la persistance des données.

Le projet est également déployé sur un VPS, dont l'adresse IP sera disponible dans le fichier `local.properties` fourni dans le code rendu. Vous pourrez ainsi tester le projet sans avoir à faire marcher le backend et reproduire nos tests durant la vidéo avec les mêmes données.

3.4.8 Maintenabilité et Évolutivité

L'architecture de NestJS nous a permis de coder de manière modulaire et standardisée, en séparant les responsabilités de chaque composant. Chaque module est indépendant et regroupe ses propres controllers, services et DTOs. De plus, NestJS intègre nativement l'injection de dépendances, ce qui permet de facilement remplacer une implémentation par une autre sans modifier le code métier, comme nous l'avons vu avec le `CdnService`.

Nous avons également appliqué les principes SOLID, notamment le principe de responsabilité unique, chaque classe a un rôle précis, par exemple un Guard ne fait que de la vérification d'accès, un Service ne fait que de la logique métier. Le principe Open/Closed est également respecté, le `CdnService` en est un bon exemple, passer à S3 ne nécessite que de modifier ce service, sans impacter le reste de l'application.

3.4.9 Gestion des FcmToken

Les FcmToken sont ce qui nous permettra de recevoir des notifications. Ci dessous un schéma de leur cycle de vie complet dans notre application :

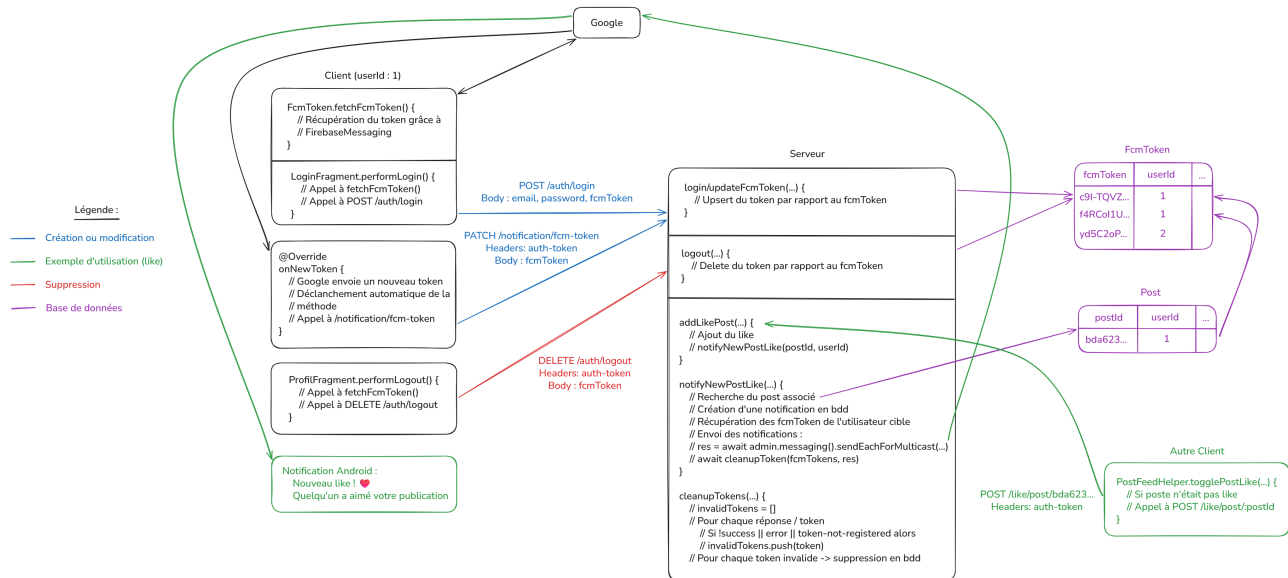


Fig. 7. – Schéma illustrant le cycle de vie complet des Fcm Tokens

La gestion du cycle de vie comme nous le faisons est la plus optimisée et correspond aux standards d'aujourd'hui. A préciser que la gestion du token dans le cache du téléphone est nativement opérée par le SDK Firebase Messaging. Le principal défi dans l'acheminement d'une notification est le stateless d'un serveur web HTTP. En effet, la seule manière serait de maintenir une connexion constante entre le téléphone et le serveur serait les web sockets, ce qui viderait sa batterie. Firebase Messaging abstrait cette complexité et nous pouvons passer par les services de Google pour remplir cette tâche, via une unique connexion TCP persistante et optimisée pour toutes les applications du téléphone. Il existe des alternatives Open Source plus libre (par exemple UnifiedPush), mais pour une première application, utiliser le standard d'aujourd'hui reste la meilleure solution.

3.4.10 Algorithme de TravelPath

Comme mentionné dans la passerelle entre TravelShare et TravelPath, TravelPath utilisera exclusivement les données de TravelShare pour construire ses parcours. Certes cette approche limite nos données disponibles mais elle nous permet de développer un algorithme fait par nos soins avec notre format de données.

Le point de départ de notre algorithme est la ville / endroit de destination, après analyse rigoureuse du sujet, nous avons décidé de ne pas inclure le trajet entre notre propre localisation la destination choisie par l'utilisateur. Le sujet mentionne explicitement « organisation du parcours de visite d'une ville ou d'un site », et les applications similaires à notre réalisation, tels que Visit A City ou Sygic Travel, distinguent (comme nous) la phase de visite local de la phase de transport en amont.

Notre algorithme se base sur une heuristique gloutonne probabiliste (d'où l'importance que notre point de départ soit la destination), soit la résolution du problème du sac à dos (heuristique pour ne pas faire exploser la complexité) avec contraintes spatiales et temporelles. Ce dernier se divise en 6 grandes étapes qui seront illustrées par le schéma ci dessous :

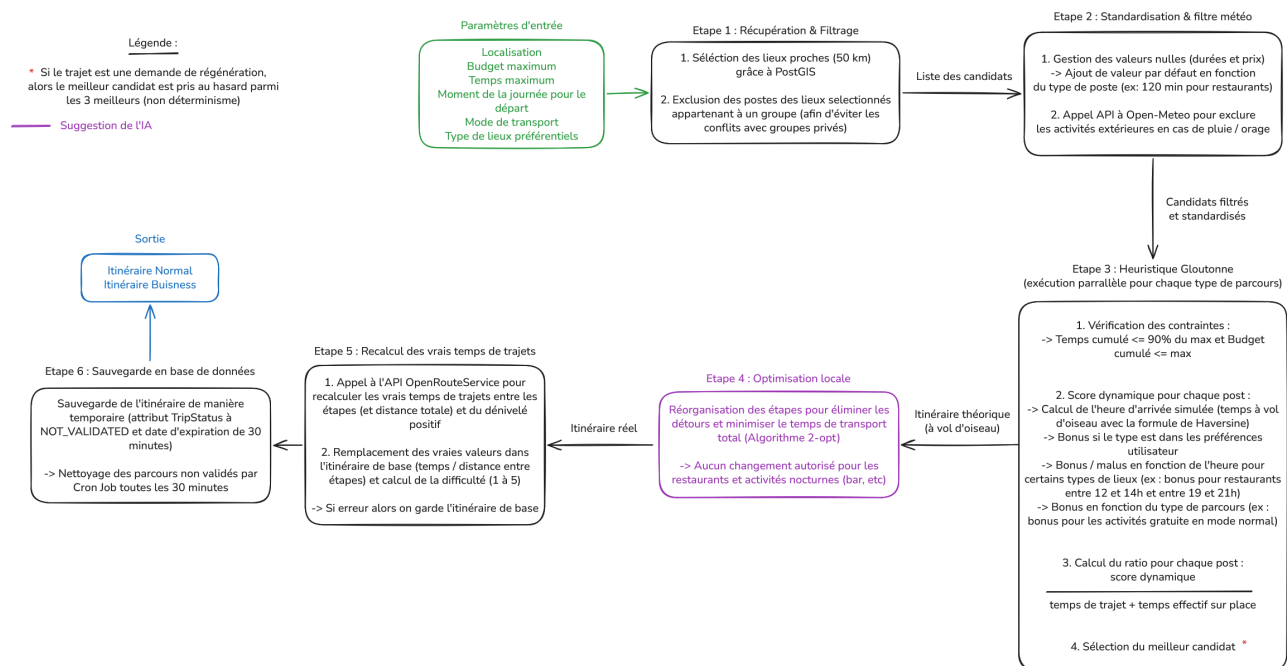


Fig. 8. – Schéma illustrant l'algorithme utilisé par TravelPath

Toutes les valeurs associées aux bonus / malus sont des constantes facilement changeables au début du fichier associé à la création de parcours, améliorant la flexibilité de l'algorithme.

Après avoir implémenté et testé notre algorithme, qui fonctionne de manière satisfaisante, nous avons demandé à l'IA de nous suggérer des améliorations, elle nous a alors proposé d'implémenter l'algorithme 2-opt [20]. Cet algorithme permet d'optimiser un problème de voyageur de commerce, évitant ainsi des trajets incohérents avec des allers retours inutiles. C'était une suggestion pertinente car elle n'apporte que de possibles améliorations à notre parcours, est facile à implémenter (en plus pas de régression) et ne fait pas exploser la complexité ($O(n^2)$ dans notre cas $n \leq 20$, donc négligeable).

Notre algorithme possède cependant un biais intrinsèque lié à sa nature gloutonne et notre scoring. Il va en effet plutôt privilégier plus d'étapes sur une moins grande distance, plutôt que d'aller assez loin pour chercher un point dans le parcours qui lui attribuera plus de points. Nous sommes conscients de ce comportement et c'est une de nos perspectives d'améliorations, nous avons pensé par exemple à autoriser plus d'exploration assuré par un mécanisme de rollback.

Nous utilisons le parallélisme de NodeJS pour calculer les deux trajets en concurrence asynchrone (Normal et Business) grâce à `Promise.all[...]`, divisant ainsi par deux le temps de réponse global.

Pour les restaurants, le score dynamique n'est pas évalué à l'heure d'arrivée sur le lieu, mais au milieu de l'activité. Ceci permet à un restaurant de ne pas obtenir le fort bonus qui lui est attribué s'il est par exemple 13h55 en heure théorique, garantissant la cohérence chronologique du parcours.

Les serveurs web étant stateless, nous avons deux manières de faire pour envoyer les parcours à l'utilisateur et s'assurer de leur authenticité (ne jamais faire confiance à l'utilisateur) après sa confirmation. L'on peut chiffrer les parcours envoyés de manière symétrique, et nous pouvons savoir si l'utilisateur a tenté de modifier les données en comparant notre version de base à sa réponse chiffrée. Soit l'on stocke tout en base de données avec un attribut de statut (VALIDATED / NOT_VALIDATED) et une date d'expiration, un cron job passera toutes les 30 minutes pour supprimer les trajets non validés. Cette

deuxième solution est meilleure, car à l'avenir, nous pourrions implémenter le meilleur des deux mondes, le cache Redis. Cette solution nous permet de stocker de manière temporaire les parcours dans une base de données en cache avant de l'écrire en vraie base de données lors de la confirmation de l'utilisateur.

4 Features non implémentées et améliorations possibles

Il n'y a pas vraiment eu d'implémentation supplémentaire autre que ce qui était demandé dans le sujet, de par la complexité de ce dernier, mais également car, comme décrit tout au long de ce rapport, les aspects non fonctionnels ont été très fortement considérés.

La seule fonctionnalité non implémentée explicitement demandée reste l'accès en hors ligne d'un parcours TravelPath, avec stockage dans le cache du téléphone. Cet aspect était à nos yeux le moins important de tous et surtout un des plus complexes. De plus l'export PDF résout la problématique de l'accès à un parcours hors ligne. Cependant cette feature manque à l'appel et nous en sommes conscients.

Notre application laisse la place à de nombreuses améliorations possible, le plus important resterait selon nous d'implémenter un stockage des images dans un bucket S3, d'améliorer encore l'algorithme de TravelPath et de mettre en place des tests unitaires pour la non régression. Une simple amélioration aurait été également d'avoir des UUID V7 et non V4 comme implémenté actuellement.

5 Conclusion

Malgré une application TravelPath difficile à imaginer, nous étions sûr du chemin à suivre et nous avons beaucoup appris. C'était notre première véritable application Android moderne, un framework backend en typescript (nous utilisons Express avant), c'était également la première fois que nous travaillions avec PostgreSQL (et PostGIS). Comme nous avons considéré les aspects non fonctionnels, nous avons beaucoup appris sur les aspects de performances, de sécurité et de scalabilité, malgré le fait que ce soit un projet universitaire.

6 Lien du dépôt GitHub

<https://urlix.me/teJySz>

Bibliographie

- [1] Kotlin, « Kotlin | Documentation », Consulté le: 18 mai 2026. [En ligne]. Disponible sur: <https://kotlinlang.org/docs/home.html>
- [2] PostgreSQL, « PostgreSQL | Documentation », Consulté le: 18 mai 2026. [En ligne]. Disponible sur: <https://postgresql.org/docs/>
- [3] PostGIS, « PostGIS | Quickstart », Consulté le: 18 mai 2026. [En ligne]. Disponible sur: https://postgis.net/documentation/getting_started/
- [4] Prisma, « Prisma | Documentation », Consulté le: 18 mai 2026. [En ligne]. Disponible sur: <https://prisma.io/docs/prisma-orm/quickstart/prisma-postgres>
- [5] Prisma Entity Relationship Diagram Generator, « prisma-erd-generator | NPM », Consulté le: 18 mai 2026. [En ligne]. Disponible sur: <https://npmjs.com/package/prisma-erd-generator>
- [6] NestJS, « Helmet | NestJS Documentation », Consulté le: 18 mai 2026. [En ligne]. Disponible sur: <https://docs.nestjs.com/security/helmet>
- [7] NestJS, « Rate limit | NestJS Documentation », Consulté le: 18 mai 2026. [En ligne]. Disponible sur: <https://docs.nestjs.com/security/rate-limiting>
- [8] NestJS, « Chiffrement | NestJS Documentation », Consulté le: 18 mai 2026. [En ligne]. Disponible sur: <https://docs.nestjs.com/security/encryption-and-hashing>
- [9] NestJS, « Validation | NestJS Documentation », Consulté le: 18 mai 2026. [En ligne]. Disponible sur: <https://docs.nestjs.com/techniques/validation>
- [10] NestJS, « File Upload | NestJS Documentation », Consulté le: 18 mai 2026. [En ligne]. Disponible sur: <https://docs.nestjs.com/techniques/file-upload>
- [11] NestJS, « Sessions | NestJS Documentation », Consulté le: 18 mai 2026. [En ligne]. Disponible sur: <https://docs.nestjs.com/techniques/session>
- [12] NestJS, « Guards | NestJS Documentation », Consulté le: 18 mai 2026. [En ligne]. Disponible sur: <https://docs.nestjs.com/guards>
- [13] NestJS, « Decorator | NestJS Documentation », Consulté le: 18 mai 2026. [En ligne]. Disponible sur: <https://docs.nestjs.com/custom-decorators>
- [14] NestJS, « Request lifecycle | NestJS Documentation », Consulté le: 18 mai 2026. [En ligne]. Disponible sur: <https://docs.nestjs.com/faq/request-lifecycle>
- [15] Wikipédia, « R-Tree | Wikipédia », Consulté le: 18 mai 2026. [En ligne]. Disponible sur: <https://en.wikipedia.org/wiki/R-tree>
- [16] NestJS, « File streaming | NestJS Documentation », Consulté le: 18 mai 2026. [En ligne]. Disponible sur: <https://docs.nestjs.com/techniques/streaming-files>
- [17] NestJS, « Error Handling | NestJS Documentation », Consulté le: 18 mai 2026. [En ligne]. Disponible sur: <https://docs.nestjs.com/exception-filters?ref=the-full-stack-engineer>
- [18] NestJS, « Deployment | NestJS Documentation », Consulté le: 18 mai 2026. [En ligne]. Disponible sur: <https://docs.nestjs.com/deployment>
- [19] Docker, « Docker Compose Quickstart | Docker Documentation », Consulté le: 18 mai 2026. [En ligne]. Disponible sur: <https://docs.docker.com/compose/gettingstarted>
- [20] Wikipédia, « Algorithme 2-opt | Wikipédia », Consulté le: 18 mai 2026. [En ligne]. Disponible sur: <https://fr.wikipedia.org/wiki/2-opt>